计算机组成与系统结构

赵超
吉林大学人工智能学院

评价标准

- **平时20%+作业20%+期末60%**
  - 4次作业，各5%，晚于Date Due提交会失去成绩
  - 平时作业与期末，王湘浩实验班与其他班会有一定区别

- **五个宽限日**
  - 作业或者签到
  - 用完后，我们不会接受任何延长时限的请求
  - 例如：Date Due 5月13日，如使用两个宽限日，5月15日23:59前提交都可以

关于这门课

- **先修课程**
  - 一种高级过程编程语言的经验，大学数学

- **课程目标**
  - 理解计算机的层次化结构与数据表示
  - 掌握指令系统与处理器工作原理理解
  - 存储层次与内存管理机制
  - 深入理解程序与操作系统的交互。

关于这门课

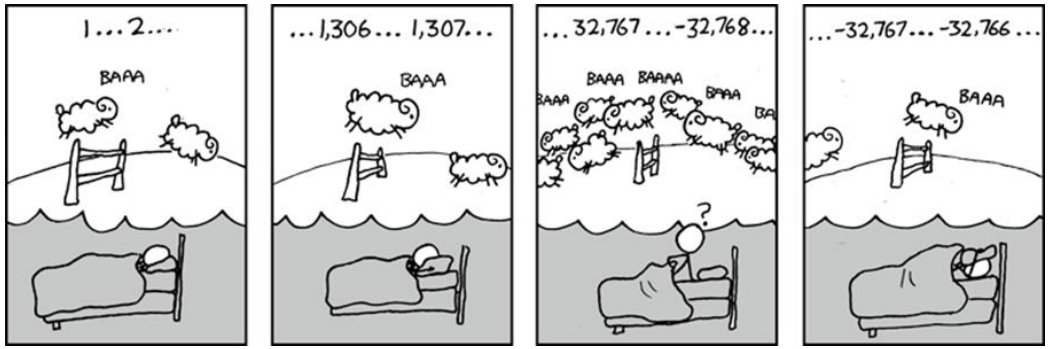
- **先修课程**
  - 一种高级过程编程语言的经验，大学数学
- **课程目标**
  - 理解计算机的层次化结构与数据表示
  - 掌握指令系统与处理器工作原理理解
  - 存储层次与内存管理机制
  - 深入理解程序与操作系统的交互。
- **简而言之：**
  - 初步涉猎如何设计一台高效的计算机
  - 初步涉猎如何让程序在计算机上高效运行

课程内容安排

- **Part0:** 系统中的数据表示和处理
- **Part1:** 指令集与处理器体系结构
- **Part2:** 存储系统与内存管理
- **Part3:** 输入输出系统
- **Part4:** 并行与多处理器，网络通信等

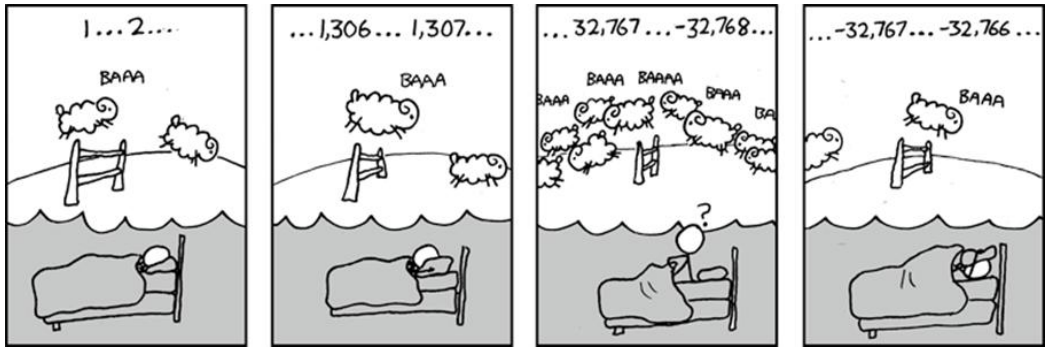
核心1：

- 例子 1：$x^2 \geqslant 0$ 吗？
  - 整数：
    - 40000 * 40000 = 1600000000
    - 50000 * 50000 = ??
    - 300 * 400 * 500 * 600 = ??



核心1：

- 例子 1：$x^2 \geqslant 0$ 吗？
  - 整数：
    - 40000 * 40000 = 1600000000
    - 50000 * 50000 = ??



- 例子 2：加法满足结合律吗？
  - (x + y) + z = x + (y + z)？
  - 浮点数：
    - (1e20 + -1e20) + 3.14 = 3.14
    - 1e20 + (-1e20 + 3.14) = ??

计算机算术运算

- **不会生成随机值**
  - 算术运算有严格的数学性质，不是随机的结果。
- **但不能假设它满足所有常见的数学性质**
  - 原因在于计算机表示的有限性。
  - 整数运算：满足“环（ring)”的代数性质
    - 交换律、结合律、分配律仍然成立
  - 浮点运算：只满足“有序性（ordering)”性质
    - 单调性、符号规则成立，但结合律/分配律可能不成立
- **启发**
  - 我们需要理解：在什么情况下这些抽象规律成立，在什么情况下会失效。
  - 这是编译器开发者和高级程序员必须认真对待的问题。

核心2：懂一点汇编（assembly）

- **也许，你这一生都不会直接用汇编写程序**
  - 现代编译器的优化能力非常强，比你更聪明、更高效、也更有耐心。
- **但是理解汇编是掌握“机器级执行模型”的关键**
  - 帮助理解程序在出现 bug 时的真实行为
    - 高级语言的抽象在复杂 bug 面前往往会“失效”，这时只有理解底层汇编，才能看懂计算机实际在做什么。
  - 优化程序性能
    - 了解哪些优化是编译器自动完成的，哪些是编译器不会做的。
    - 理解性能瓶颈的根源。
  - 应对安全与恶意软件
    - x86 汇编都是安全领域的首选语言。

核心3：内存很重要

- **随机访问存储器（RAM）并不是一个真正“无限、均匀”的概念**
  - 内存不是无限的
    - 必须经过分配和管理
    - 很多应用程序都是受限于内存的
  - 内存性能并不均匀
    - 缓存和虚拟内存的特性会显著影响程序性能
    - 如果根据内存体系的特点优化程序，性能可能会获得极大提升
  - 内存引用错误尤其棘手
    - 错误的影响跨越时间和空间，可能很久之后才显现

内存引用错误示例

```c
typedef struct {
    int a[2];
    double d;
} struct_t;

double fun(int i) {
    volatile struct_t s;
    s.d = 3.14;
    s.a[i] = 1073741824; /* 可能越界访问 */
    return s.d;
}
```

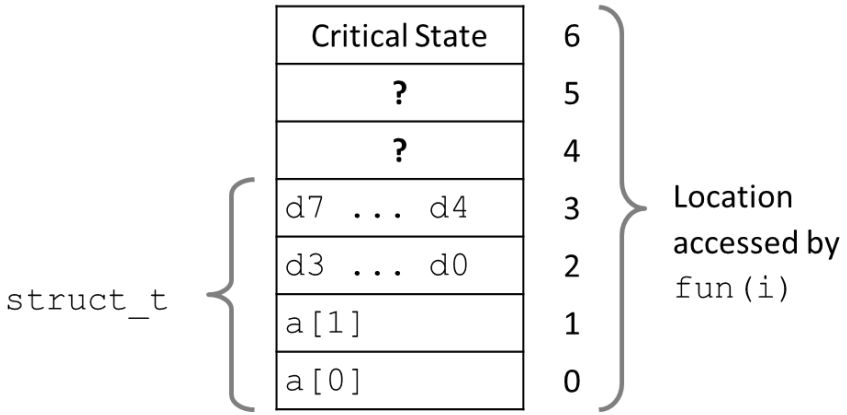
- 运行结果示例：

```
fun(0) -> 3.14
fun(1) -> 3.14
fun(2) -> 3.139998664856
fun(3) -> 2.00000061035156
fun(4) -> 3.14
fun(6) -> 程序崩溃（Segmentation fault）
```

内存引用错误示例

```c
typedef struct {
    int a[2];
    double d;
} struct_t;

double fun(int i) {
    volatile struct_t s;
    s.d = 3.14;
    s.a[i] = 1073741824; /* 可能越界访问 */
    return s.d;
}
```



- 运行结果示例：

```
fun(0) -> 3.14
fun(1) -> 3.14
fun(2) -> 3.139998664856
fun(3) -> 2.00000061035156
fun(4) -> 3.14
fun(6) -> 程序崩溃（Segmentation fault）
```

- 结果是系统相关的，不同平台表现可能不同。

内存引用错误总结

- C / C++ 不提供任何内存保护
  - 可能出现数组越界访问
  - 无效指针引用
- 这些问题会导致严重的 Bug
  - Bug 的影响常常取决于系统和编译器
  - 错误可能出现在“很远的地方”，让人难以定位
  - 数据损坏可能在很久之后才表现出来
- 如何应对？
  - 使用带内存保护的语言，例如 Python 等
  - 理解底层数据结构的内存布局，预防潜在错误
  - 使用工具，如 Valgrind，自动检测内存引用错误

核心4：性能不仅仅取决于算法复杂度

- **常数因子同样重要！**
  - 在算法分析中，我们通常关注时间复杂度，但在实际编程中，常数因子也会极大影响性能。
- **即使精确的操作计数，也不能预测程序性能**
  - 同样的操作数量，代码写法不同，性能可能相差 10 倍。
  - 必须在多个层面上优化：算法、数据表示、函数过程、循环结构等。
- **必须理解系统，才能真正优化性能**
  - 了解程序是如何被编译和执行的。
  - 学会如何衡量程序性能并识别性能瓶颈。
  - 在不破坏代码模块化和通用性的前提下，提升性能。

内存系统性能示例

```
void copyij(int src[2048][2048],
            int dst[2048][2048])
{
  int i,j;
  for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
      dst[i][j] = src[i][j];
}
```

```
void copyji(int src[2048][2048],
            int dst[2048][2048])
{
  int i,j;
  for (j = 0; j < 2048; j++)
    for (i = 0; i < 2048; i++)
      dst[i][j] = src[i][j];
}
```

4.3ms **81.8ms**

2.0 GHz Intel Core i7

- 现代计算机采用分层式内存组织。
- 程序性能高度依赖访问模式：
  - 如果访问是连续的，缓存命中率高，性能大幅提升。
  - 如果访问是跳跃的，缓存失效频繁，性能显著下降。
- 多维数组的遍历顺序会直接影响缓存效率。

核心5：计算机的世界远不止于运行程序

- **计算机需要与外界交换数据**
  - 数据输入与输出（I/O）是任何计算机系统的核心功能。
  - I/O 系统的设计和性能直接影响程序的可靠性和运行速度。
  - 如果 I/O 设计不当，即使计算速度再快，也可能被数据传输瓶颈拖慢。
- **并行与多处理器的挑战**
  - 现代计算机通常包含多个处理器核心，支持并行计算以提高性能。但并行并不意味着简单地“越多越快”
- **网络通信**
  - 在今天的世界里，计算机几乎从不孤立运行，而是通过网络协作完成更复杂的任务。

课程内容安排

- **Part1:** 系统中的数据表示和处理
- **Part2:** 指令集与处理器体系结构
- **Part3:** 存储系统与内存管理
- **Part4:** 输入输出系统
- **Part5:** 并行与多处理器，网络通信等

1:系统中的数据表示和处理

1.1比特和比特运算

1:系统中的数据表示和处理

1.1比特和比特运算

- 数字就是数字?
- 两个正数相乘一定是正数?
- (A+B=C；A+C=D)A+C一定开始于A+B之后吗?

overflow-demo — terminal

```
$ cat OverflowDemo.java
public class OverflowDemo {
    public static void main(String[] args) {
        int a = 2147483647;
        int b = 1;
        int x = 50000;
        int y = 50000;

        System.out.println(a + " + " + b + " = " + (a + b));
        System.out.println(x + " * " + y + " = " + (x * y));
    }
}

$ java_
```

overflow-demo — terminal

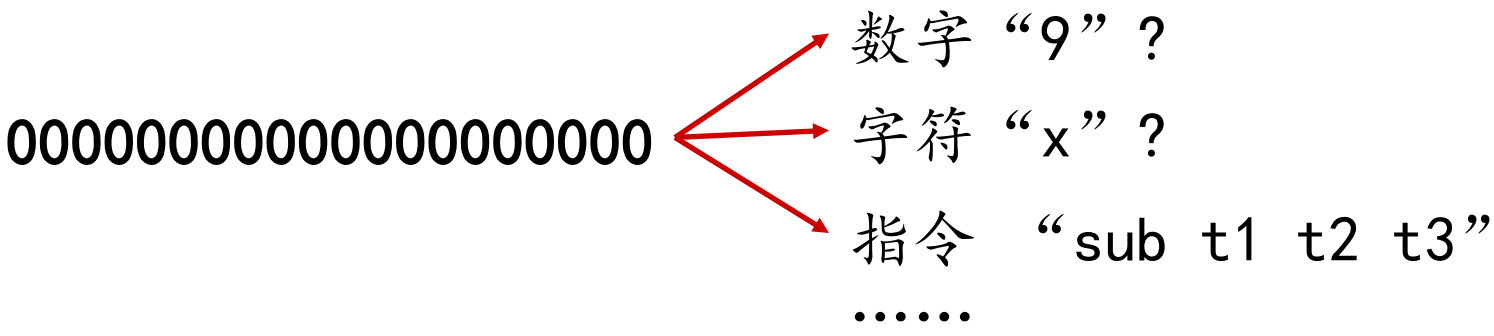
_

比特/位 (Bits)

- 比特值为 0 或 1

比特/位 (Bits)

- 比特值为 0 或 1
- 通过以不同方式编码/解释一组比特
  - 计算机确定执行的操作（指令）
  - … 并表示和操作数字、集合、字符串等



宾夕法尼亚大学第一台电子计算机



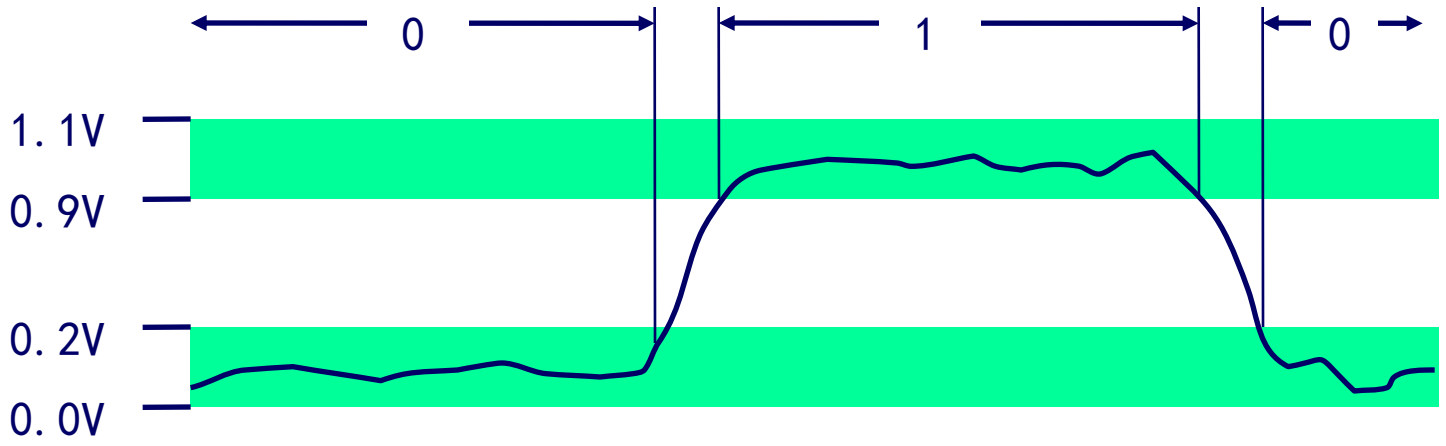
中国第一台电子计算机 103



- 研制成功1958年8月
- 科研人员在资源匮乏的情况下，凭借坚韧不拔的精神，克服困难，实现了中国计算机领域的突破。
- 当代青年肩负起国家科技创新的责任，为实现中国梦贡献力量。

比特/位 (Bits)

- 比特值为 0 或 1
- 通过以不同方式编码/解释一组比特
  - 计算机确定执行的操作（指令）
  - … 并表示和操作数字、集合、字符串等
- 为什么使用比特?
  - 电气上易于储存，表示
  - 可在嘈杂且不准确的线路上可靠传输



回顾：二进制计数

- 二进制数表示法
  - 15213_{10} 表示为 11101101101101_2
  - 1.20_{10} 表示为 $1.00110011001100110011[0011]\cdots_2$
  - 1.5213×10^4表示为 $1.1101101101101_2 \times 2^{13}$
  - IEEE二进制浮点数算术标准（IEEE 754）
- 计算机表示数字时不是直接输入我们写在代码中的样子，而是按二进制规则重新编码。

字节值编码

- **Byte = 8 bits**
  - 2进制(Binary)：00000000_2 to 11111111_2
  - 10进制(Decimal)：0_{10} to 255_{10}
  - 16进制(Hexadecimal)：00_{16} to FF_{16}
    - 使用字符 '0' 到 '9' 和 'A' 到 'F'
    - $FA1D37B_{16}$ 在 C 语言中表示为：
      - 0xFA1D37B
      - 0xfa1d37b

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

1:系统中的数据表示和处理

1.1比特和比特运算

布尔代数 (Boolean Algebra)

- George Boole 在19世纪提出
  - 逻辑的代数表示
    - 将 “True” 编码为 1, “False” 编码为 0

布尔代数（Boolean Algebra）

- George Boole 在19世纪提出
  - 逻辑的代数表示
    - 将 “True” 编码为 1， “False” 编码为 0

- 既然逻辑可以用“True”与“False”来表示，能否设计一套代数系统，通过定义一组结构化操作，捕捉人们在逻辑推理中自然假设的一些概念？

布尔代数 (Boolean Algebra)

- George Boole 在19世纪提出
  - 逻辑的代数表示
    - 将 “True” 编码为 1， “False” 编码为 0

与 And

- 当A=1 并且 B=1时，A&B = 1

&	0	1
0	0	0
1	0	1

或 Or

- 当A=1 或 B=1时A|B = 1

|	0	1
0	0	1
1	1	1

非 Not

- 当A=0时，~A = 1

~	
0	1
1	0

异或 (Xor)

- 当A=1 或 B=1且两者不同时为1， A^B = 1

^	0	1
0	0	1
1	1	0

布尔代数 (Boolean Algebra)


克劳德·香农(1916-2001)

- 核心洞察：开关电路可以看作逻辑系统。
  - 继电器的“通/断”可以对应逻辑中的“真/假”。由此，电路状态可以用 0 和 1 表示。
- 这使抽象的逻辑代数能够直接指导实际电路设计。为整个现代计算机科学和信息论奠定了基础。
- 所以其硕士论文被认为是人类历史上最具影响力的硕士论文之一。

布尔代数（Boolean Algebra）

- 把布尔运算应用到一串比特上，按比特/位运算
- 位向量操作
  - 与（&），或（|），异或（^），非（~）
  - 按比特/位运算
- 布尔代数的全部性质均适用

```
  01101001        01101001        01101001
& 01010101      | 01010101      ^ 01010101      ~ 01010101
----------      ----------      ----------      ----------
  01000001        01111101        00111100        10101010
```

布尔代数（Boolean Algebra）

- 把布尔运算应用到一串比特上，就叫按位运算
- 位向量操作
  - 与（&），或（|），异或（^），非（~）
  - 按位运算
- 布尔代数的全部性质均适用

```
  01101001        01101001        01101001
& 01010101      | 01010101      ^ 01010101      ~ 01010101
----------      ----------      ----------      ----------
  01000001        01111101        00111100        10101010
```

- 布尔运算在计算机实践中证明是有用的，作为一种隐式的表示数值集合的方式。

比特运算与集合的表示

- 集合表示
  - 宽度 w 个比特的向量表示集合 {0, …, w–1}的子集
  - $a_j = 1$ if $j \in A$
    - 01101001 { 0, 3, 5, 6 }
    - *76543210*

比特运算与集合的表示

- 集合表示
  - 宽度 w 个比特的向量表示集合 {0, …, w–1}的子集
  - $a_j = 1$ if $j \in A$
    - 01101001 { 0, 3, 5, 6 }
    - *76543210*
    - 01010101 { 0, 2, 4, 6 }
    - *76543210*

比特运算与集合的表示

- 集合表示
  - 宽度 w 个比特的向量表示集合 {0, …, w–1}的子集
  - $a_j = 1$ if $j \in A$
    - 01101001 { 0, 3, 5, 6 }
    - *76543210*
    - 01010101 { 0, 2, 4, 6 }
    - *76543210*
- 集合运算

&	交集	01000001	{ 0, 6 }
|	并集	01111101	{ 0, 2, 3, 4, 5, 6 }
^	对称差集	00111100	{ 2, 3, 4, 5 }
~	补集	10101010	{ 1, 3, 5, 7 }

位集合在 I/O 状态中的应用

示例：用 fd_set 跟踪哪些文件描述符“已经准备好读取数据”。

文件/网络连接

每个 fd 对应位图中的一位

fd0	键盘输入	ready=0
fd1	本地文件	ready=0
fd2	**Socket A**	**ready=1**
fd3	Socket B	ready=0
fd4	**Pipe**	**ready=1**
fd5	日志文件	ready=0
fd6	**Socket C**	**ready=1**
fd7	空闲	ready=0

状态写入位图

隐藏数据结构：fd_set

一个比特就能表示一个 I/O 状态；系统可用按位运算一次处理多个 fd。

fd编号	7	6	5	4	3	2	1	0
	0	1	0	1	0	1	0	0

集合值：01010100 → { fd2, fd4, fd6 } 已就绪

& 交集

哪些 fd 同时在“监听集合”和“就绪集合”中

| 并集

把多个监听集合合并成一个位集合

~ 补集

排除已经处理过的 fd，留下未处理项

核心思想：集合运算可以转化为 CPU 很擅长的按位操作，所以适合管理大量 I/O 状态。